

MMIO Access-Based Coverage for Firmware Analysis

Ken (Yihang) Bai
University of Florida
baiyihang@ufl.edu

Tuba Yavuz
University of Florida
tuba@ece.ufl.edu

Abstract—Firmware rehosting techniques have enabled the analysis of firmware without a real device through peripheral access modeling. However, existing techniques use the number of basic blocks to evaluate success in firmware code coverage without paying attention to whether code locations that process data received from specific peripherals receive sufficient coverage. In this paper, we present a hybrid firmware analysis approach, FIRMSTAT, that combines symbolic execution based firmware rehosting and static code analysis to evaluate dynamic code coverage in terms of the API functions that propagate data register values into the upper layers of firmware. Our static analysis supports both polling-based and interrupt-driven accesses and extract API summaries. We apply FIRMSTAT to various ARM Cortex M3 benchmarks from the literature and show that despite demonstrating a decent performance in terms of basic block coverage compared to some of the state-of-the-art firmware rehosting approaches, it may fail to cover some peripheral data-flows in complex firmware, which can be captured by the proposed MMIO access-based coverage metrics.

Index Terms—firmware rehosting, symbolic execution, static analysis

I. INTRODUCTION

Firmware is typically formed of multiple layers including the application layer, the third-party library layer, the Hardware Abstraction Layer (HAL), and the device driver layer. Firmware developers may rely on the correctness of third-party libraries or misuse their APIs, which may result in various types of vulnerabilities that can be exploited through attacker controlled inputs received from the peripherals. Therefore, peripheral interactions form part of the attack surface in firmware, and, hence, it is important to identify how data received from the peripherals flow to different layers of the firmware.

Firmware rehosting allows emulation of firmware on a host device. However, the complexity of control logic in the device driver functions prevents high coverage in complex firmware. Although rehosting approaches can potentially be used to find firmware vulnerabilities, the generated peripheral access models may not allow for achieving meaningful coverage regarding peripheral accesses.

One of the goals of firmware rehosting approaches is the ability to analyze binary firmware. Although this goal is important for reverse engineering and vetting unknown firmware, there are settings where firmware analysis can

assume the availability of the firmware source code. For instance, advanced security testing of firmware during code development or the development of sophisticated intrusion detection systems that involve firmware analysis are scenarios where we can make that assumption.

In this paper, we argue that although basic block coverage is a useful metric, it does not reveal much information for the settings that we consider in this paper. We present a symbolic execution based firmware rehosting and a static analysis based MMIO access relevant API model extraction to provide MMIO relevant coverage of firmware emulation. To this end, we have developed FIRMSTAT¹ on top of S2E and SVF. Our results on firmware benchmarks show that state-of-the-art basic block coverage is not a good indicator of high coverage of code that process data received from the peripherals.

This paper is organized as follows. In Section II we present the challenges we tackle in firmware analysis. We present the details of our path exploration heuristics and static analysis based MMIO access models in Section III. We present evaluation results in Section IV. We position our work within the context of related work in Section V. We conclude with directions for future work in Section VI.

II. OVERVIEW

In this section, we provide an overview of our approach, FIRMSTAT, which handles different challenges of symbolic execution of firmware while trying to provide coverage information in terms of functions that access peripheral generated data values. Similar to other approaches such as [1], our firmware rehosting approach uses symbolic values for the data received from the peripherals, i.e., for loads from the Memory Mapped I/O (MMIO) addresses.

The challenges that FIRMSTAT focuses on can be summarized as follows:

Challenge 1: Unbounded Forking: If a branch condition is symbolic and if it appears in a loop either as the loop condition or another potentially unbounded loop, new states will be forked as long as the branch condition does not violate the path condition. This leads to a path explosion, and, ultimately, new coverage will degrade.

¹FIRMSTAT will be released at <https://github.com/sysrel/FIRMSTAT>.

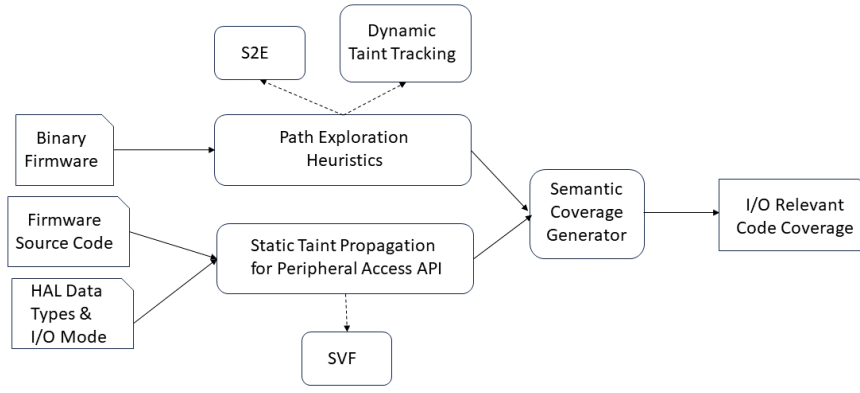


Fig. 1. Architecture of the FIRMSTAT framework

Challenge 2: Interrupt Space Complexity: Firmware is a piece of event-driven software, where the peripherals may generate interrupts to notify the arrival of data or its availability to handle a command. If interrupts are not triggered at the right time, interrupt-driven firmware will get blocked or keep executing the same set of instructions that check the status changes in the hardware. Generating all possible interrupts and frequently does not scale as it leads to an uncontrollable path explosion.

Challenge 3: Generic Coverage Metrics: Firmware typically uses a library or a real-time operating system that provides a Hardware Abstraction Library (HAL) that interacts with the peripherals. HAL API calls the driver API, which handles the low-level communication with the peripherals within the context of a specific board. Data read from the peripherals within these HAL API are then propagated to the upper layers of the firmware. Current approaches use the number of basic blocks covered to measure coverage. Although being a useful coverage metric, it does not give an idea on how much of the peripheral data-flows are covered during firmware rehosting.

FIRMSTAT has been designed as shown in Figure 1 to deal with the above challenges. Change in code coverage is used as an indicator for progress and is used to decide whether any intervention is needed for firmware execution. FIRMSTAT uses the knowledge of enabled interrupts and MMIO access models to optimize interrupt execution. Finally, it implements dynamic tracking for recording MMIO access contexts and static analysis to leverage the knowledge of MMIO programming model to extract MMIO access API models to provide new coverage metrics related to the reads from the peripheral data registers.

III. APPROACH

A. Symbolic Execution Based Rehosting

Our symbolic execution based firmware rehosting approach consists of four major components: coverage manager, fork manager, interrupt manager, and taint tracker. Coverage manager evaluates the progress of a symbolic execution state in

two alternative modes: window-based and no window based. In window-based mode, for each state the set of basic blocks are recorded for the last several windows, where each window is represented by the execution of a preset number of basic blocks. Coverage manager makes scheduling decisions as well as invoking of the fork manager by comparing the most recent preset number of windows. In no window mode, those decisions are made based on the coverage and the forking events happened in the last preset number of basic blocks. We explain the other three components in the following subsections.

1) *Fork Manager:* We model peripheral interactions by representing the results of the read operations involving the MMIO address ranges with unconstrained symbolic values. In symbolic execution, if the branch condition is a symbolic expression, the executor will consider both cases and generate a symbolic execution state/path for each feasible target, whose branch condition is satisfiable. Some of the symbolic values that represent MMIO read accesses end up flowing into various control-flow decisions, which may lead to a path explosion if not controlled. This forking behavior gets exacerbated when the branch instruction corresponds to checking of a loop condition. Figure 2 shows a code example that involves a symbolic loop condition. The available PIN numbers ($pinNumber$) is read from some MMIO region (line 2) and this symbolic value becomes the loop bound value (line 3) to initialize each PIN (line 4). After forking the first time, two states will be generated with opposite branch conditions. One state will continue the second iteration of the loop body (line 4) with the condition $1 < pinNumber$ and the other one will break the loop with the branch condition $1 \geq pinNumber$ and jump outside (next line after line 5). If the scheduler chooses the former state as the next explored state, it will continue to perform the second fork (line 3) with two opposite conditions, $2 < pinNumber$ and $2 \geq pinNumber$. This process might continue forever if no action is taken by the symbolic executor, and the path explosion will block us from reaching code locations beyond the loop and from achieving higher code coverage.

To demonstrate the difficulty of the problem, consider the statement at line 6 in Figure 2 that checks the symbolic value *pinNumber*. The result of the condition determines whether the function returns a success or a failure value. Although one may be tempted to have the symbolic executor arbitrarily stop forking at the loop condition after a fixed number of iterations, such states, despite making some progress in terms of achieving coverage early on, will fail to continue doing so due to unmet conditions later in the execution, e.g., returning a false value for *initPin* will cause the while loop at line 10 continue executing the same loop body.

```

01. bool initPin() {
02.     int pinNumber = getAvailablePinNum();
03.     for (int i = 0; i < pinNumber; i++) {
04.         pinInitialize(pinTable[i]);
05.     }
06.     return pinNumber > 6 : true : false;
07. }
08.
09. void miscInit(){
10.     while( !initPin() );
11.     ...
12. }

```

Fig. 2. An example with multiple fork points.

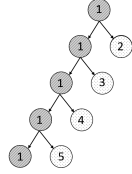


Fig. 3. An example symbolic execution tree for repeated forking at some forking point.

To address the above issues, we implement a fork manager (FM), and present the details in Algorithm 1. At the time of invoking FM, we have a set of candidate states, *CS*, e.g., the states labeled 1-5 in the execution tree shown in Figure 3, where State 1 forks several times at the same fork point and generates states 2-5. When the coverage manager detects such repeated forking pattern, it will transfer control to FM and states 1-5 will be collected as the set of candidate states. The fork points for these branches are recorded in a set called OFP. Once FM takes control of the execution engine, it will be provided with one original execution state and several execution states forked at the same fork point, marked as the original fork points (OFPs)², e.g., line 3 in Figure 2.

The OFP is structured as the concatenation of a summary of the call stack information and the program counter (PC) of the forking instruction. As an example, in Figure 4, if *r3* at 0x80077de (line 15) contains a symbolic value and the state keeps forking at 0x80077e0 (line 16), the OFP would be as shown in Figure 5, where for each function in the call stack its entry point and the return address get concatenated. To have a sense of meaningful progress in terms of coverage, we introduce the concepts of target end exploration point (TEEP)

²Sometimes there will be more than one repeated fork point.

Algorithm 1 The algorithm for the Fork Manager (FM).

```

1: ForkManager(CS: Candidate States, TD: TEEP Depth, OFP: Original Fork Points, FLM: Fork Limit Map, MFT: Max Fork Times)
2: Output (TS: TargetState)
3: FLM : ExecutionState × OFP → Num    ▷ Fork Times at each OFP
4: CR : ExecutionState → Coverage        ▷ CoverageRecorder
5: TSS ← ∅
6: CandidateStatesSet : DT1, DT2, DT3
7: for State in CS do
8:     State.StopReason = NULL
9:     while true do
10:        pc, forked, NewState ← executeNextInstruction()
11:        CR[state].insert(pc)
12:        if forked then
13:            context ← getCurrentForkContext()
14:            if context in OFP then
15:                if FLM[state][context] > MFT then
16:                    State.StopReason = REACH_MAX_FL
17:                else
18:                    FLM[state][context] ++
19:                    FLM[NewState] = FLM[state]
20:                    CS.insert(NewState)
21:                end if
22:            else
23:                State.StopReason = REACH_NEW_FP
24:                DT1.insert(State)
25:            end if
26:        else
27:            context ← OFP(state)
28:            if hitKillPointOrInvalidState(pc) then
29:                State.StopReason = REACH_KP_IS
30:            else if hitAgainLEEP(context, pc, TEEPDepth) then
31:                State.StopReason = RE_REACH_END
32:                DT3.insert(State)
33:            else if (pc == getTEEP(context, TEEPDepth)) then
34:                State.StopReason = REACH_TARGET_END
35:                DT2.insert(State)
36:            end if
37:        end if
38:        pc ++
39:        if State.stopReason! = NULL then
40:            break
41:        end if
42:    end while
43: end for
44: TSS ← SelectNotEmptyStateSetInOrder(DT1, DT2, DT3)
45: TS ← highestCoverageInTSS(CR, TSS)
46: return TS

```

```

01. function_1<0x8001234>:
02.     ...
03.     8001254: bl function_2 <0x8002068>
04.     8001258: mov r1, r0
05.     ...
06.
07. function_2<0x8002068>:
08.     ...
09.     800207a: bl function_3 <0x80077ca>
10.     800207e: ldr r3, [r7, #16]
11.     ...
12.
13. function_3<0x80077ca>:
14.     ...
15.     80077de: cmp r3, #0
16.     80077e0: bne.n 0x80077d0
17.     ...

```

Fig. 4. Example to illustrate the OFP representation.

and low-level end exploration points (LEEP). The points are defined based on the function return addresses. For example,

in Figure 5, if we define 0x8001258 as the TEEP, then all the return addresses before TEEP returns are in LEEP, i.e. 0x800207e. We define the TEEP in a given OFP through the TEEP Depth (TD) parameter to denote the number of functions to be completed to reach the TEEP.

In addition to the candidate set of states, Algorithm 4 takes as input the TEEP Depth (TD), and a fork limit map (FLM), which represents the number of times a candidate state has forked at some OFP. The algorithm can be configured by setting the maximum number of times a state can fork at the same OFP, which is called MFT.

All the candidate states, CS , will be collected by FM and they will be scheduled alternatively until all of them have been explored. When FM has the control over scheduling, each state needs to be either stashed or terminated for meeting different conditions to yield the execution to the other candidate states. This allows exploration of all or most of the candidate states within a limited time by FM. The conditions that stop the exploration of the current execution state when under the control of FM are as follows:

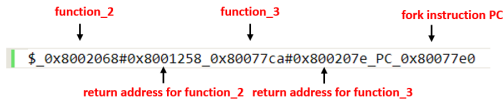


Fig. 5. An example for TEEP and LEEP.

- Execution meets a new fork point (lines 22-25), different from the OFP, e.g. in Figure 2 OFP consists of the branch on line 3 and the one on line 6 is considered as a new fork point. Such states are stashed and recorded in DT_1 .
- Execution returns from the current function context for a given call stack depth, e.g., for the function *initPin* in Figure 2, if the execution state could return from this function and also the outer layer function *miscInit*, i.e., reaching the TEEP, we need to hold this state and try other states (lines 33-36). Such states are stashed and recorded in DT_2 .
- There has been repeated returning from the current function context without reaching the TEEP, i.e., reaching a LEEP second time (lines 30-32). For instance, a function with a boolean return type might be wrapped within a loop condition, e.g., line 10 in Figure 2, to wait until the function returns the desired value. If the execution tries to return from the same function the second time, we need to make it yield to other execution states. Such states are stashed and recorded in DT_3 .
- The execution encounters a user defined kill point or an execution error, e.g., if *Error_handler* is reached, we need to switch to another state (lines 28-29). Such states are terminated.

With such termination or yielding conditions, the exploration of all previously collected states could be finished in practical time. Each explored state will be handled based

on the type of condition meets first. Furthermore, for each execution state, we record the number of times the state has forked at the original OFPs. If the value exceeds a preset number MFT , the execution state is terminated. Otherwise, the new forked state will be pushed to the candidate states for further exploration. This avoids the generation of an unbounded number of states during the exploration. Finally, after all the states have been scheduled, we select one state from these states based on how they get stashed, i.e., according to the order: DT_1 , DT_2 and DT_3 , as indicated on line 43 in Algorithm 1. To identify the final target state, we select the state that has the highest coverage among the target states set, TSS, (line 44). If there are multiple states with the same coverage, we choose the one that appeared the latest.

With FM, we are able to handle path explosion in a configurable way. In addition, we provide an option to generate a loop bound model after each state exiting from FM. It analyzes the MMIO accesses related to the OFPs, solves the symbolic constraints to concretize values within the selected state and provides the concrete values at the same MMIO location when the same function gets executed the next time. The goal of this option is to avoid the repeated analysis of the same forking context. This option is based on two assumptions. First, the MMIO accesses yielding an unbounded number of forks is near the fork point. Second, our state strategy selects states that have reached code locations of interests, i.e., new OFPs or the TEEP, that are likely to yield high coverage later on (lines 44-45).

2) *Interrupt Manager*: In firmware rehosting, interrupts must be explicitly triggered by the emulation as there is no real hardware to generate them. Without a timely generation of the interrupts, it is not possible to achieve meaningful coverage as the execution will stop making progress either due to waiting on a lock/semaphore or continuously checking some condition in a loop as shown in Figure 6 on line 14. Previous re-hosting works such as [1] injects interrupts in a round-robin fashion. These firmware analysis tools trigger interrupts based on a preset interval, e.g., after the execution of every 2000 basic blocks. We follow the practice of uEmu [1] to explore the IRQ handler with symbolic execution and trigger some interrupt every 1000 basic blocks, each time we execute the IRQ handler in concrete mode and use concrete MMIO value models. Qemu provides a virtual interrupt controller that we can use to trigger the interrupt. After triggering the interrupt line, the executor will be directed to the IRQ handler function, on which we apply symbolic execution. However, it is still possible to have numerous paths inside the IRQ handler and it is impractical to decide which path is desired.

A challenge in triggering interrupts is that there are many of those, and triggering all in symbolic execution mode would lead to a path explosion. Our interrupt triggering policy deals with this challenge in the following way. Every time the executor detects an interrupt line being enabled, it keeps a back-up state of the current execution state and pauses the execution. Then we invoke the interrupt manager (IM) and

initialize a new state pool. IM triggers the enabled interrupt line, and starts to explore the interrupt handler function using symbolic execution only for the first time to learn a peripheral access model for the interrupt handler. If there are other new forked states generated during the process, we push it to the state pool. We apply a simpler version of the Fork Manager and set a limit on the number of times a state can fork at each Program Counter (PC). The executor continues executing the same state until the state is stashed or terminated according to the following rules:

- A state exits the interrupt handler (likely).
- A state forks many times and reaches the maximum forking limit (likely).
- A state is terminated by the kill points (unlikely).
- A state is in arbitrary execution, e.g. executes outside the ROM (unlikely).

```

01.  bool flag;
02.
03.  void ISR_peripheralA() {
04.      ...
05.      int status = getPeripheralAStatus();
06.      flag = status > 6 : true : false;
07.      ...
08.  }
09.
10.  void peripheralATransmit() {
11.      ...
12.      flag = false;
13.      peripheralASend();
14.      while (!flag);
15.      ...
16.  }

```

Fig. 6. An example for managing interrupts.

After exploring all the states that get generated inside the interrupt handler, we stop the IM and extract MMIO access models from all the explored states by solving the constraints generated within the interrupt handler and recording it as a sequence of MMIO address and value pairs. So, for the example in Figure 6, IM analyzes *ISR_peripheralA* (lines 3-8) and generates two models for the status of the peripheral that is accessed at line 5. In the first model the MMIO address will be associated with a value greater than 6 and in the second model it will be associated with a value smaller than or equal to 6. Since both models will be used in a round robin fashion, the *flag* variable will eventually be set to true at line 6, and the execution will be able to exit the loop on line 14.

Once we extract all the MMIO access models for the interrupt line, we exit from IM and restart from the backup state. So far, we have all the MMIO access models for the specific interrupt line and we do not explore it using symbolic execution the second time. When the executor returns to the normal mode (Coverage Manager), we start to trigger this explored interrupt line using the round-robin approach. For the second time of triggering an interrupt line and onward, we feed the interrupt handler with one of the MMIO access

models that have been generated when it was triggered the first time.

3) *Dynamic Taint Tracking*: Taint tracking of symbolic expressions happen automatically in symbolic execution as when some memory location holding a symbolic expression is read and copied onto a new memory location the symbolic expression is copied without any concretization. So, we leverage this feature of symbolic execution to track where the symbolic MMIO values propagate, except in the case of MMIO access models generated as part of interrupt handling. We use hooks for memory access operations to record specific memory locations that get tainted. Although branching on a symbolic value is a natural way to detect such tainting, it is possible that a symbolic expression may not ever get used in a branch condition. So, the hooks allow us to track all symbolic expressions that are not concretized for optimization purposes.

B. Static Taint Analysis

The Hardware Abstraction Layer (HAL) provides a set of API to communicate with the peripherals for performing I/O. In this section, we explain variations on the HAL API with regards to reading data from the peripherals and present a static analysis algorithm to extract a taint model.

Firmware may perform I/O using polling or interrupts. In the polling approach, the firmware continuously checks for some status register to find out whether the data is available and reads from the data register when it gets available. In the interrupt-driven approach, the firmware designates an interrupt handler and a receive buffer, which is used by the interrupt handler to store the incoming data. After enabling the interrupts, the firmware either busy waits or blocks until the interrupt handler finishes its job of storing the data in the receive buffer. Then the firmware finds out the completion and processes the data in the receive buffer.

Figure 7 shows the data types and the data-flow paths that involve input data registers. Based on our analysis of open-source device drivers for embedded systems, we find a common pattern for both the data structures and the mechanisms involved with some slight variations. Typically, there is a device type that is defined as a struct, where the device registers are defined as the fields. Another data type, the handle type, keeps a pointer to an instance of the device type and defines fields that represent the receive buffer, *RXBuffer*, and the interrupt handler for the input/receive events, *RXISRCallback*, among other things. We consider two modes of I/O: polling-based and interrupt-driven. Since we are interested in how the implementation works, we further divide interrupt-driven I/O implementation into two subclasses: with or without a *RXISRCallback* in the handle type. In the latter case, a specific API is designated as the interrupt handler, and applications are expected to redefine that function. In the former case, the applications can use an existing interrupt setup API to set the interrupt handler. In

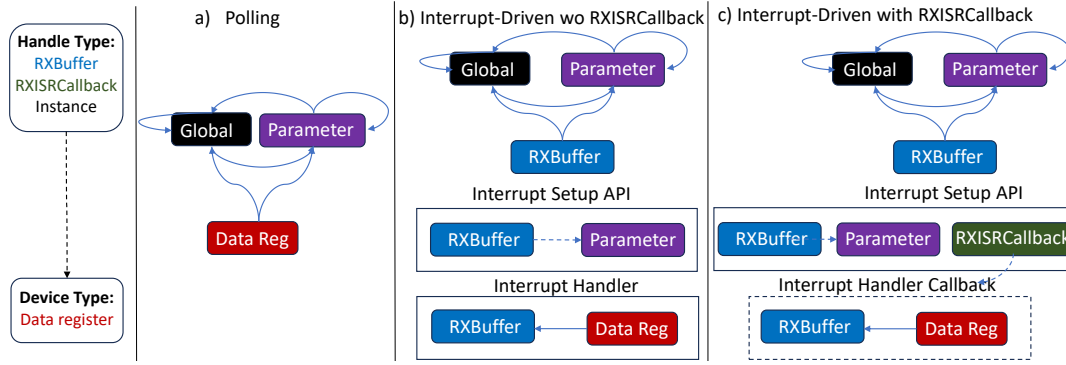


Fig. 7. Various types of data flows from the peripheral data registers to the upper layers. Solid arrows represent data flows and dashed arrows represent points-to relationship.

both cases, the pointer to the `RXBuffer` gets assigned to a pointer type parameter or to the address of a global variable.

We have designed a static analysis algorithm that gets the device data type, the data register field, the handle type, the `RXBuffer` field, and the `RXISRCallback` field. Algorithm 2 presents the extraction of API taint models of the data registers for interrupt-driven I/O with `RXISRField` at a high-level, and leave out the algorithms for polling-based and interrupt-driven without the `RXISRCallback` field due to space restrictions. The algorithm keeps track of the function parameters, *TFA*, and the global variables, *GV*, that get tainted. It handles direct as well as indirect callsites. It initializes these global variables based on direct flows from the data register as well as direct assignments to the receive buffers. Then it updates these in the **Step** function to reflect new function pointer resolutions based on *FDT* and *CSCB* and new tainted global variables and function arguments. The main algorithm, **FindMMIODataFlows** computes a fixpoint until no new tainted global variables, arguments, and implicit calls to the tainted functions can be found. Finally, it returns the tainted global variables and the function arguments.

IV. EVALUATION

We have implemented FIRMSTAT using S2E for ARM Cortex M3 as the symbolic execution based emulator and SVF [2] as the static analyzer. We have applied FIRMSTAT to P2IM firmware benchmarks and to the AWS Jobs demo. We evaluated FIRMSTAT in terms of code coverage compared to *uemu*'s symbolic execution component.

Table I shows a comparison of *uEmu-learning*, which uses symbolic-execution based emulation, and FIRMSTAT on a subset of P2IM benchmarks. Code coverage is measured in terms of the number of basic blocks executed. As the table shows, in 8 out of 10 cases FIRMSTAT achieves more coverage compared to *uEmu*. Then we compared FIRMSTAT with *uEmu* and Fuzzware on AWS Jobs demo. Fuzzware uses fuzzing and on-demand symbolic execution for peripheral access model generation. Fuzzware has been shown to achieve improved coverage over state-of-the-art firmware analysis approaches including *uEmu*. We ran *uEmu-learning* for more than 9 hours and it covered 634 basic blocks. We ran

TABLE I
COMPARISON OF FIRMSTAT AND *uEmu* WITH RESPECT TO CODE COVERAGE MEASURED IN TERMS OF THE NUMBER OF BASIC BLOCKS.

Benchmark	uEmu-learning	FIRMSTAT	
		Window	No Window
CNC	574	713	693
Console	627	597	431
Drone	1373	1381	902
Gateway	769	1077	714
Heat_press	435	467	445
PLC	334	380	429
ReflowOven	830	822	897
Robot	976	1013	611
Soldering_iron	1438	879	1146
Steering_Control	385	362	403

Fuzzware on the AWS Jobs demo for 24 hours and it covered 813 basic blocks. FIRMSTAT, on the other hand, covered 1338 basic blocks in an hour.

Given that FIRMSTAT achieves better or comparable coverage to state-of-the-art firmware rehosting approaches, we measured the coverage of API functions with tainted parameters or with access to tainted global variables. We utilized both dynamic taint analysis that we implemented as part of firmware rehosting and static analysis that we implemented on top of SVF as shown in Figure 7. Our static MMIO access API model extraction finished within 5 mins for both case studies. We used the symbols extracted from the firmware ELF file to decide the MMIO relevant API functions that are intended to be used in the firmware.

We performed two case studies on the benchmarks Drone and AWS JobsDemo [3] implementation for the STM32L475 board. Tables II-III and IV-V show the results. For the Drone benchmark, we analyzed accesses to the data registers of the I2C and UART peripherals. For the AWS JobsDemo, we analyzed the accesses of the SPI, UART, and RNG peripherals.

We list the functions in which peripheral accesses happen according to dynamic taint tracking (implemented as part of our firmware rehosting approach) and according to static taint tracking. The Drone benchmark uses I2C in polling mode. As Table II shows, static analysis identifies seven APIs whereas dynamic analysis

Algorithm 2 Extracting API taint models of the data registers for interrupt-driven I/O with RXISRField.

```

1: Global  $TFA, FDT, CSCB, GV$ 
2:
3: FindMMIODataFlows( $M$ : Module,  $DT$ : Device Type,  $FD$ : Data Register Field,  $HT$ : Handler Type,  $FB$ : Buffer Field,  $ISR$ : ISR Field)
4: Init( $M, DT, FD, HT, FB, ISRF$ )
5: while true do
6:    $(TFA', FDT', CSCB', GV') \leftarrow (TFA, FDT, CSCB, GV)$ 
7:   Step( $M, DT, FD, HT, FB, ISRF$ )
8:   if  $TFA' = TFA$  and  $FDT' = FDT$  and  $CSCB' = CSCB$  and  $GV' = GV$  then
9:     break
10:  end if
11: end while
12: return( $TFA, GV$ )
13:
14: Init( $M$ : Module,  $DT$ : Device Type,  $FD$ : Data Register Field,  $HT$ : Handle Type,  $FB$ : Buffer Field,  $ISRF$ : ISR Field)
15:    $\triangleright$  Find candidate interrupt setup API
16:  $ISRS \leftarrow \{f \mid \exists si \in M.Store. s.t. Dest(si) \text{ is of type } (HT, ISRF) \wedge Source(si) = f\}$ 
17:    $\triangleright$  Find direct flows from the data register
18:  $DirectFlow \leftarrow \{si \mid \exists si \in M.Store. s.t. Value(si) \text{ is of type } (DT, FD)\}$ 
19:    $\triangleright$  Find global variables tainted due to direct flows
20:  $GV \leftarrow \{gv \mid si \in DirectFlow. Dest(si) = gv\}$ 
21:    $\triangleright$  Find tainted function arguments due to direct flows
22:  $TFA \leftarrow \{(f, ArgNo(a)) \mid \exists f \in M.Func. si \in DirectFlow. a \in FormalArg(f). s.t. si \in f.Store \wedge Value(si) = a\}$ 
23:    $\triangleright$  Find tainted function arguments due to tainted local vars passed to interrupt setup API
24:  $TFA \leftarrow TFA \cup \{(f_1, ArgNo(a)) \mid \exists f_1 \in M.Func. f_2 \in ISRS. cs \in f_1.Callsites. si \in f_1.Store. l \in f_1.Locals. s.t. Called(cs) = f_1 \wedge l \in Arg(cs) \wedge (f_2, ArgNo(l)) \in TFA \wedge Dest(si) \text{ is of type } (HT, FB) \wedge Value(si) = l\}$ 
25:
26: Step( $M$ : Module,  $DT$ : Device Type,  $FD$ : Data Register Field,  $HT$ : Handle Type,  $FB$ : Buffer Field,  $ISRF$ : ISR Field)
27:    $\triangleright$  Find function pointer types associated with functions with tainted arguments
28:  $FDT \leftarrow \{(type, field, f) \mid \exists si \in M.Store. (f, a) \in TFA. s.t. Value(si) = f \wedge Dest(si) \text{ is of type } (type, field)\}$ 
29:  $\cup \{(type, field, f) \mid \exists cs \in M.Callsites. (f, a) \in TFA. a' \in Arg(cs). s.t. a' = f \wedge FormalArg(cs) \text{ is of type } (type, field)\}$ 
30:    $\triangleright$  Associate indirect function calls related to tainted function pointer types with the target functions
31:  $CSCB \leftarrow \{(cs, f) \mid \exists cs \in M.Callsites. (type, field, f) \in FDT. s.t. cs \text{ is a function pointer of type } (type, field)\}$ 
32:    $\triangleright$  Mark global variables passed as arguments of tainted function
33:  $GV \leftarrow \{gv \mid \exists cs \in M.Callsites. (f, a) \in TFA. a' \in Arg(cs). s.t. Called(cs) = f \wedge a' = gv \wedge ArgNo(a') = a\}$   $\triangleright$  Mark functions that call tainted functions and pass their args to the tainted args as tainted
34:  $TFA \leftarrow \{(f', ArgNo(a')) \mid \exists cs \in f'. Callsites. s.t. Called(s) = f \wedge a' \in f'.FormalArg \wedge a' \in Arg(cs)\}$ 
35:
36:    $\triangleright$  Mark functions with direct callsites receiving tainted global variables as arguments as tainted
37:    $TFA \leftarrow TFA \cup \{(f, ArgNo(a')) \mid \exists gv \in GV. cs \in M.Callsites. a' \in Arg(cs). s.t. f = Called(cs) \wedge a' = gv\}$ 
38:
39:    $\triangleright$  Mark functions with indirect callsites receiving tainted global variables as arguments as tainted
40:    $TFA \leftarrow TFA \cup \{(f_1, ArgNo(a')) \mid (cs, f_1) \in CSCB \wedge \exists gv \in GV. a' \in Arg(cs). s.t. gv = a'\}$ 

```

TABLE II

I2C DATA REGISTER ACCESS BASED COVERAGE OF THE DRONE FIRMWARE AND COMPARISON OF DYNAMIC AND STATIC TAINT ANALYSIS RESULTS. - MEANS DYNAMIC TRACKING DID NOT DETECT THE DATA REGISTER ACCESS. * MEANS THE FUNCTION GOT EXECUTED DURING SYMBOLIC EXECUTION. COVERAGE OF REHOSTING BASED ON EXECUTION OF I2C RELEVANT API AND BASED ON I2C DATA REGISTER ACCESS ARE BOTH 4 OUT OF 5.

Taint Tracking	
Dynamic	Static
HAL_I2C_Mem_Read	HAL_I2C_Mem_Read
I2C_ReadByte	I2C_ReadByte
-	I2C_ReadBytes
I2C_ReadByteArray	-
MS5611_ReadADC	MS5611_ReadADC
-	HAL_I2C_Slave_Receive
-	HAL_I2C_Master_Receive
-	HAL_I2C_EV_IRQHandler

identifies four APIs. Static analysis fails to identify the I2C_ReadByteArray as FIRMSTAT currently do not track the return values of MMIO accessing functions. Two of the three APIs, HAL_I2C_Slave/_Master_Receive,

TABLE III

UART DATA REGISTER ACCESS BASED COVERAGE OF THE DRONE FIRMWARE AND COMPARISON OF DYNAMIC AND STATIC TAINT ANALYSIS RESULTS. * MEANS THE FUNCTION GOT EXECUTED DURING SYMBOLIC EXECUTION. COVERAGE OF REHOSTING BASED ON EXECUTION OF UART RELEVANT API AND BASED ON UART DATA REGISTER ACCESS ARE 6 OUT OF 7 AND 1 OUT OF 7, RESPECTIVELY.

Taint Tracking	
Dynamic	Static
- (*)	UART_Receive_IT
- (*)	HAL_UART_Receive_IT
-	HAL_UART_Receive
- (*)	HAL_UART_IRQHandler
- (*)	HAL_UART_RxCpltCallback
- (*)	USART1_IRQHandler
serialBegin	serialBegin
-	HAL_UART_Receive_DMA

are to be used in polling mode, but it turns out that they are not used in the Drone benchmark. The other one, HAL_I2C_EV_IRQHandler, is related to interrupt handling. So, we think that there are a total of five API functions that are related to receiving input from I2C. FIRMSTAT was

TABLE IV

MMIO ACCESS BASED COVERAGE OF THE AWS JOBS DEMO FIRMWARE AND COMPARISON OF DYNAMIC AND STATIC TAINT ANALYSIS RESULTS. - MEANS DYNAMIC TRACKING DID NOT DETECT THE DATA REGISTER ACCESS. * MEANS THE FUNCTION GOT EXECUTED DURING SYMBOLIC EXECUTION. COVERAGE OF REHOSTING BASED ON EXECUTION OF SPI RELEVANT API AND BASED ON SPI DATA REGISTER ACCESS ARE 6 OUT OF 30 AND 0 OUT OF 30, RESPECTIVELY.

Taint Tracking	
Dynamic	Static
-(*)	HAL_SPI_Init
-(*)	SPI_WIFI_MspInit
-	SPI_2linesRxISR_16BIT
-	SPI_2linesRxISR_8BIT
-	SPI_RxISR_16BIT
-	SPI_RxISR_8BIT
- (HAL_SPI_IRQHandler*)	-
-(*)	HAL_SPI_Receive
-(*)	HAL_SPI_TransmitReceive
-	HAL_SPI_Receive_IT
-	HAL_SPI_TransmitReceive_IT
-	ES_WIFI_GetNetworkSettings
-	ES_WIFI_StartClientConnection
-	ES_WIFI_StopClientConnection
-	ES_WIFI_ReceiveData
-	ES_WIFI_ReceiveDataFrom
-	ES_WIFI_StoreCA
-	ES_WIFI_StoreCertificate
-	ES_WIFI_StoreKey
-(*)	ES_WIFI_Init
-	AT_ExecuteCommand
-	AT_RequestSendData
-	ES_WIFI_Connect
-	ES_WIFI_IsConnected
-	ES_WIFI_Disconnect
-	ES_WIFI_Ping
-	ES_WIFI_DNS_LookUp
-	ES_WIFI_SendData
-(*)	WIFI_On
-	WIFI_Reset

TABLE V

MMIO ACCESS BASED COVERAGE OF THE AWS JOBS DEMO FIRMWARE AND COMPARISON OF DYNAMIC AND STATIC TAINT ANALYSIS RESULTS. - MEANS DYNAMIC TRACKING DID NOT DETECT THE DATA REGISTER ACCESS. * MEANS THE FUNCTION GOT EXECUTED DURING SYMBOLIC EXECUTION. COVERAGE OF REHOSTING BASED ON EXECUTION OF RNG RELEVANT API AND BASED ON RNG DATA REGISTER ACCESS ARE BOTH 0 OUT OF 3.

Taint Tracking	
Dynamic	Static
-	HAL_RNG_IRQHandler
-	HAL_RNG_GenerateRandomNumber
-	HAL_RNG_GetRandomNumber

able to achieve %80 coverage both in terms of execution of these APIs and detecting MMIO reads.

The Drone benchmark uses the UART in interrupt mode. As Table III shows, static analysis identifies eight functions, one of them being DMA related and not used by the firmware. Although FIRMSTAT executed 6 out of 7 functions that access the data register of UART, it didn't detect the accesses to the data register, yielding a lower coverage of MMIO accesses (14%) compared to coverage of APIs (86%). This

may be due to the symbolic execution paths ending before reaching the code locations that read the data register as such functions typically first access some control or status registers before reading the received data.

Table IV shows the results for the SPI data register for the AWS Jobs Demo. SPI uses interrupt-driven I/O. FIRMSTAT found 29 APIs that access the SPI data register and missed one, HAL_SPI_IRQHandler, which uses an indirect call to the RXISR_Callback. However, FIRMSTAT was not able to access the data register during symbolic execution. It should be noted that Fuzzware achieved much less coverage than FIRMSTAT on this benchmark as we mentioned earlier, and if FIRMSTAT could not read from the SPI data register, it is very unlikely that Fuzzware would access it. In terms of execution of the SPI relevant API, FIRMSTAT achieved 20% coverage. This shows that the WIFI component is complicated and can benefit from modeling of some of the low level APIs.

For the UART data register, FIRMSTAT found ten relevant APIs. However, a close inspection of the executed functions and the binary symbols shows that, AWS Jobs Demo uses UART for sending out data. So, a coverage metric for this peripheral turned out not to be applicable for this peripheral.

Finally, for the RNG data register, FIRMSTAT found three relevant APIs as shown in Table V. However, FIRMSTAT could neither execute the APIs nor access the data register.

Our evaluation shows that FIRMSTAT can achieve a decent coverage according to the number of basic blocks metric and compared to some of the state-of-the-art tools. However, we also show that, despite achieving good basic block coverage, MMIO data register access coverage may be low for certain peripherals, especially for the AWS Jobs Demo. So, we think that basic block coverage should be supplemented with the MMIO access based coverage metrics we presented in this paper to better evaluate the behavioral coverage of firmware.

V. RELATED WORK

FIRMADYNE [4] uses full system emulation for dynamic analysis of Linux-based embedded firmware from different architectures and shows the feasibility of scaling vulnerability detection for a large set of firmware images based on known exploits. The challenges of analyzing embedded firmware that runs on arbitrary real-time operating systems or board support packages and interact with a variety of on-chip and off-chip peripherals led to the investigation of hardware-in-the-loop approaches such as PRETENDER [5], which learns a peripheral model by combining firmware emulation with real hardware and recording the MMIO responses and interrupt events from the device. The learnt model is used in rehosting firmware. Despite the advantages of using real MMIO responses, pure rehosting approaches has the obvious benefit of handling multiple device types and providing an easier setup. Early rehosting approaches [6], [7] dealt with the complexity of peripheral interactions through modeling. HALucinator [6] performs high level emulation of firmware by replacing Hardware Abstraction Layer (HAL) API with manually prepared

models using contextual binary matching techniques. P2IM [7] presents a pattern-based extraction of concrete peripheral and firmware I/O models to achieve firmware rehosting. Both HALucinator and P2IM demonstrate the effectiveness of their rehosting approaches through fuzzing, which succeeds in finding vulnerabilities in the analyzed firmware. Our approach focuses on the automated modeling of HAL and other functions based on the propagation of values read from the data registers.

To deal with the implicit peripheral interactions through DMA, DICE [8] detects DMA channels used by firmware and generates more paths compared to analyses that cannot leverage DMA channels. This approach is complementary to our approach and the knowledge of accesses through DMA can improve the precision of MMIO access based coverage.

Laelaps [9] uses symbolic execution for firmware rehosting. It switches between symbolic execution and concrete execution for scalability, and uses context-bounded symbolic execution to generate consistent concrete models of peripheral accesses. Jetset [10] performs targeted symbolic execution by progressively generating an updated control-flow graph to steer the execution towards the goal. It generates a mostly symbolic peripheral access model as the path explosion is controlled by filtering paths based on the target. μ Emu [1] generates a multi-tier peripheral access model using symbolic execution and associates the concrete values with increasingly detailed context information to achieve precision. The generated models are used during fuzzing. Our approach achieves better or comparable basic block coverage compared to μ Emu on the P2IM benchmarks by relying on symbolic execution only. FIRMSTAT can also report MMIO access based coverage when the firmware source code is available.

Fuzzware [11] uses locally-scoped dynamic symbolic execution to identify the parts of the peripheral data that affects the firmware logic. Despite its outstanding performance over the state-of-the-art firmware rehosting approaches, it may still fail to analyze complex firmware. We think that both Fuzzware and FIRMSTAT can benefit from guidance based on MMIO propagation models across various layers of the firmware.

FIRMSTAT comes with new heuristics such as fork limiting and bounded symbolic execution of interrupt handlers to deal with the path explosion problem in symbolic execution of firmware. Existing firmware rehosting approaches focus on improving code coverage measured in terms of the number of basic blocks executed. FIRMSTAT, on the other hand, leverages the existence of source code, which may be feasible in various security relevant contexts, and presents MMIO access based coverage as a new metric to evaluate success.

VI. CONCLUSIONS

We present FIRMSTAT, which generates statically extracted MMIO data register API models and uses novel symbolic path exploration heuristics to achieve improved code coverage. FIRMSTAT can report MMIO access based

coverage by combining static and dynamic taint tracking. Our results show that basic block coverage should be supplemented with MMIO access based coverage when source code of the firmware is available. In future work, we will incorporate MMIO access models to guide the path exploration heuristics.

ACKNOWLEDGEMENTS

This work was funded by the US National Science Foundation under the CNS-1942235 award.

REFERENCES

- [1] W. Zhou, L. Guan, P. Liu, and Y. Zhang, "Automatic firmware emulation through invalidity-guided knowledge inference," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/zhou>
- [2] Y. Sui and J. Xue, "SVF: interprocedural static value-flow analysis in LLVM," in *Proceedings of the 25th International Conference on Compiler Construction, CC 2016, Barcelona, Spain, March 12-18, 2016*, A. Zaks and M. V. Hermenegildo, Eds. ACM, 2016, pp. 265–266.
- [3] "Amazon freertos," <https://github.com/aws/amazon-freertos>.
- [4] D. D. Chen, M. Woo, D. Brumley, and M. Egele, "Towards automated dynamic analysis for linux-based embedded firmware," in *23rd Annual Network and Distributed System Security Symposium, NDSS 2016, San Diego, California, USA, February 21-24, 2016*. The Internet Society, 2016. [Online]. Available: <http://wp.internetsociety.org/ndss/wp-content/uploads/sites/25/2017/09/towards-automated-dynamic-analysis-linux-based-embedded-firmware.pdf>
- [5] E. Gustafson, M. Muench, C. Spensky, N. Redini, A. Machiry, Y. Fratantonio, D. Balzarotti, A. Francillon, Y. R. Choe, C. Kruegel, and G. Vigna, "Toward the analysis of embedded firmware through automated re-hosting," in *22nd International Symposium on Research in Attacks, Intrusions and Defenses, RAID 2019, Chaoyang District, Beijing, China, September 23-25, 2019*. USENIX Association, 2019, pp. 135–150. [Online]. Available: <https://www.usenix.org/conference/raid2019/presentation/gustafson>
- [6] A. A. Clements, E. Gustafson, T. Scharnowski, P. Grosen, D. Fritz, C. Kruegel, G. Vigna, S. Bagchi, and M. Payer, "Halucinator: Firmware re-hosting through abstraction layer emulation," in *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, S. Capkun and F. Roesner, Eds. USENIX Association, 2020, pp. 1201–1218.
- [7] B. Feng, A. Mera, and L. Lu, "P2IM: scalable and hardware-independent firmware testing via automatic peripheral interface modeling," in *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020*, S. Capkun and F. Roesner, Eds. USENIX Association, 2020, pp. 1237–1254. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/feng>
- [8] A. Mera, B. Feng, L. Lu, and E. Kirda, "Dice: Automatic emulation of dma input channels for dynamic firmware analysis," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 1938–1954.
- [9] C. Cao, L. Guan, J. Ming, and P. Liu, "Device-agnostic firmware execution is possible: A concolic execution approach for peripheral emulation," in *Annual Computer Security Applications Conference, ser. ACSAC '20*. New York, NY, USA: Association for Computing Machinery, 2020, p. 746–759.
- [10] E. Johnson, M. Bland, Y. Zhu, J. Mason, S. Checkoway, S. Savage, and K. Levchenko, "Jetset: Targeted firmware rehosting for embedded systems," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021.
- [11] T. Scharnowski, N. Bars, M. Schloegel, E. Gustafson, M. Muench, G. Vigna, C. Kruegel, T. Holz, and A. Abbasi, "Fuzzware: Using precise MMIO modeling for effective firmware fuzzing," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 1239–1256.